



TRITON VENTURES • THE TECHNICAL READ

How it actually works

*The architecture, the protocol, the security floor, and the tests — for a room that reads diagrams the way other rooms read headlines. **Prepared for Bill, Zak, and Mike.***



HOW TO READ THIS

Start where you already stand

MIKE • SYSTEMS

You simplify the process *before* you automate it. So the first three slides are the process, not the code — **what talks to what, and over what wire.**

ZAK • VOCABULARY

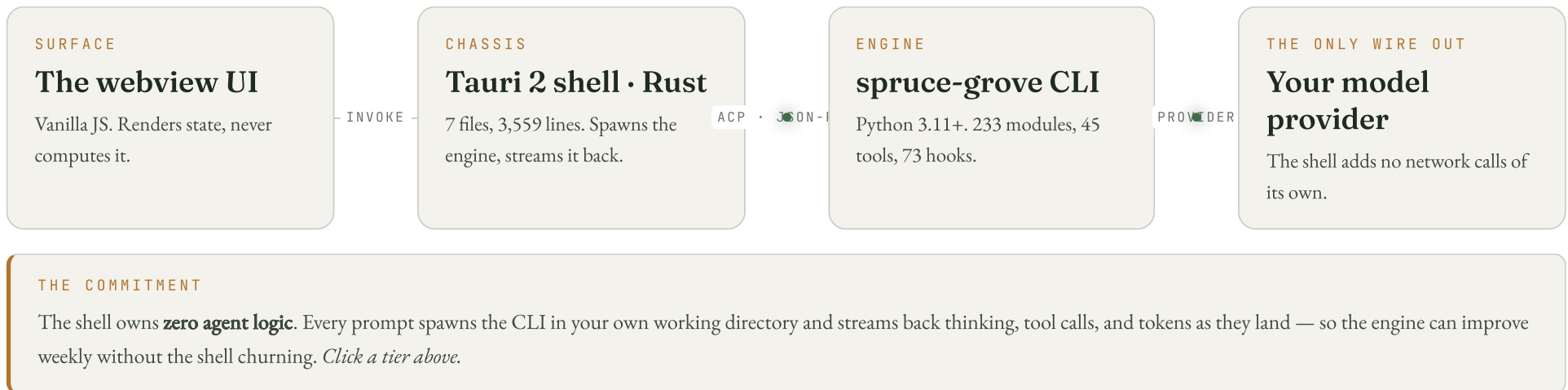
Harness engineering, loop engineering, sovereign AI. This deck is the **substrate under that vocabulary** — 73 hook phases, a local loop, a swappable engine.

BILL • DURABILITY

The question never changes: *is the moat real?* Slides 11–13 are the answer — **a security floor and a test matrix that fail the build on drift.**

THE SHAPE • CLICK ANY TIER

Thin shell, tested engine, one wire



THE ENGINE • SPRUCE-GROVE

A harness, not a wrapper

233 PYTHON MODULES	45 TOOLS	30 AGENTS
73 HOOK PHASES	356 TEST FILES	0 TELEMETRY CALLS

THE LOOP

Pydantic-AI underneath: typed tool schemas, structured outputs, retries with checkpoints.

Providers are swappable — OpenAI, Anthropic, Gemini, or a local endpoint on your own metal.

THE CONTEXT

Sessions are files, not a server. Resume, compact, and a **turn-scoped working set** so a long session does not drag its whole history into every call.

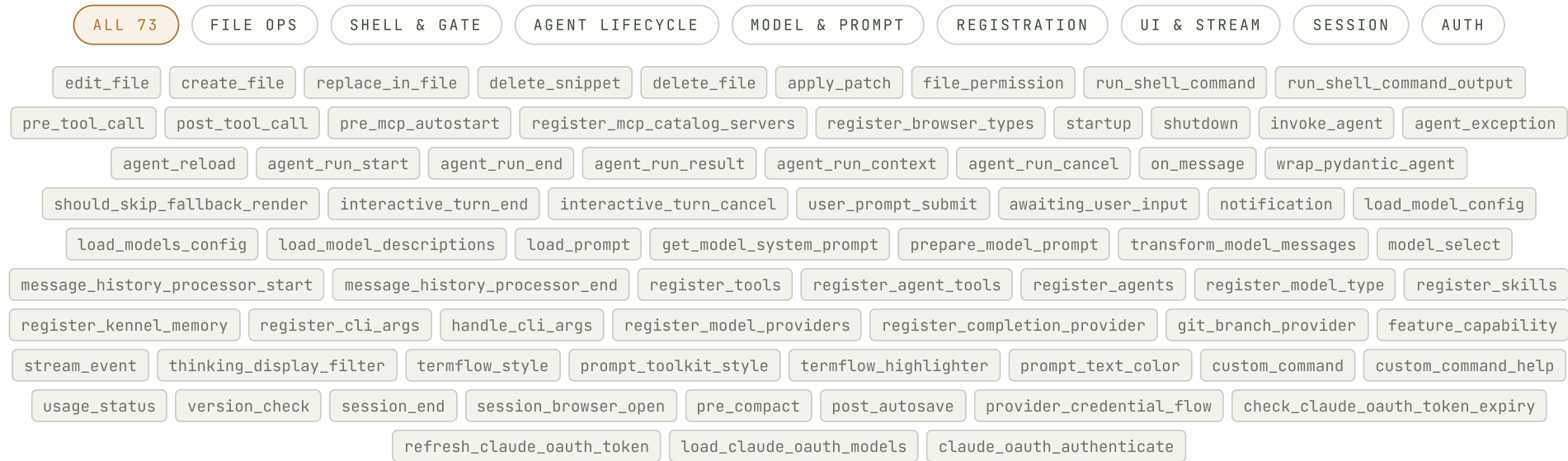
THE CONSCIENCE

Self-heal, diagnostics, a judge agent, and a **ledger that writes down every decision** — including the ones that turn out wrong.

NO INVENTED FACTS — IF A NUMBER CAN DRIFT, COMPUTE IT

THE EXTENSION SEAM • CLICK A CLASS

73 hook phases, one bus



WHY THIS IS THE MOAT, NOT A FEATURE

Every capability is a callback on a named phase — never an edit to core. Three tiers load in order (**builtin** → **user** → **project**), project wins collisions, and a crashed guard on a security phase **fails closed**. *Hover a phase to light its neighbours.*

THE PLUGIN MODEL

Extend it without forking it

TIER	LIVES AT	WHO OWNS IT	TRUST
builtin	code_puppy_core_plugins/	the project	discovered via entry points
user	~/ .spruce_grove/plugins/	you, on every project	your machine, your call
project	<repo>/ .spruce_grove/plugins/	the repo, shared via git	SHA-256 gated — nothing loads until you type "trust"

THE DETAIL THAT MATTERS

Project-plugin trust is a hash of the directory, scoped to that project path. **Any file change reverts to untrusted.** Runtime state is banned from the plugin dir — because state next to the code would self-tamper the hash. Accepted plugins hot-load with no restart; `/plugins revoke` pulls trust.

7 files. 3,559 lines. 37 commands.

MODULE	LINES	JOB
<code>recordings.rs</code>	832	the recording library — play, splice, lock, export
<code>acp.rs</code>	713	the protocol router — classify, route, stream, cancel
<code>inspector.rs</code>	705	branch · commits · tree · PRs · CI, read live
<code>main.rs</code>	619	the 37 <code>#[tauri::command]</code> entry points
<code>project.rs</code>	405	governance docs found by convention, not a list
<code>audit.rs</code>	147	the self-audit floor — CSP + capability surface
<code>ledger.rs</code>	138	the append-only action ledger

Everything above streams through `invoke`; only live updates ride events. That split is deliberate — and it is why a missing event permission once made the app "look stuck streaming" forever.

JSON-RPC, not a private dialect

*The shell and engine speak **ACP** — the same protocol other clients speak. That is what makes the engine separable: the desktop is one face, and a terminal, a CI job, or another editor can be another.*

INITIALIZE

Protocol version negotiated up front, then capabilities — **including the command list the UI renders.**

SESSION/NEW • SESSION/LOAD

A session survives process death: a fresh agent recalls the marker.
Proven, not assumed — the probe test kills the process and asks.

PROMPT • UPDATE

A turn streams back as chunk, thought, tool-call, and turn-end notifications — **classified by the router, never string-matched in the UI.**

PERMISSION

An agent request becomes an allow-kind prompt; the option chosen is deterministic and **unit-tested.**

A RECORDED SESSION REPLAYS THROUGH THE REAL ROUTER ON EVERY RUN — PROTOCOL DRIFT FAILS FIRST

ONE TURN · STEP THROUGH IT

The life of a prompt

COMPOSE

SPAWN

INJECT

MODEL

TOOL

PERM

STREAM

LEDGER

STEP 1 · COMPOSE

You type, or you speak. The prompt lands in the shell, which never parses it — it hands the turn to the engine whole. No pre-processing to diverge from what you meant.

◀ BACK

NEXT ▶

VOICE • ON-DEVICE

Speak it, and the audio stays home

*Mic → local Whisper → an editable prompt. The audio never leaves the machine. But the interesting engineering is the **guard against a model that confidently invents sentences.***

THE FAILURE

Whisper hallucinates on silence — repeating stock phrases like *"Thank you."* across a pause. A naive transcript reads as real speech.

DENSITY, PER SECOND OF SPEECH

Silence is measured with `ffmpeg silencedetect` at `-30 dB, 0.5 s` min, and **subtracted** from duration. The guard is `characters ÷ speech seconds, floor 2.0`.

STOCK-TOKEN RESIDUE

Every known stock phrase is stripped, then the remainder is checked. Repeated *"Thank you."* leaves nothing — and dies. **Real speech that says "thank you" survives.**

FAILS SAFE

If silence detection fails, it falls back to total duration — so the guard **can only get less strict, never more.**

27 TESTS • REAL TAKES, REAL PAUSES, A REAL HALLUCINATION THAT STILL GETS REJECTED

THE FLOOR • TOGGLE IT OFF AND WATCH IT FAIL

Security the build refuses to lose

CSP

object-src 'none' – no plugin/object execution

CSP

base-uri 'none' – no injected base tag rewriting URLs

CSP

form-action 'none' – no exfiltrating via a rogue form post

CSP

frame-ancestors 'none' – cannot be framed/clickjacked

CAP

capability surface locked to one window ("main")

NET

shell adds zero network calls – provider traffic is the only wire

SIMULATE DRIFT ►

RESTORE ◀

THE MECHANISM

These are not settings someone remembers to check. `audit.rs` and the UI suite both assert the same list; **if either drifts, the test suite goes red and the release is blocked.** Press *"simulate drift"*.

THE PROOF • TESTED AT THE SEAMS

Four layers, each catching a different lie

LAYER	WHAT IT RUNS	THE LIE IT CATCHES
the contract	a recorded ACP session replayed through the real router	protocol drift — fails loudly, first
the engine	356 test files, 8,119 tests, on every push; macOS before a release	a tool, hook, or session regression the shell would inherit
the chassis	39 Rust tests; CI builds macOS + Windows + Ubuntu	a platform seam that only breaks on someone else's machine
the glass	a Chromium suite driving the real UI JS, network blocked	a UI that works mocked and dies live
the floor	CSP + capability assertions in runtime <i>and</i> tests	a security default quietly lowered to make a demo pass

Onboarding, the project tracker, doc-convention discovery, and the one-file showcase packet all have their own suites. None of it is aspirational — it runs.

SOVEREIGNTY • OBSERVABLE BY DESIGN

If it can drift, it is written down

THE LEDGER

An append-only action log. Inspect it in-app; it is the same record the tests read. **Decisions, not vibes.**

THE INSPECTOR

Branch, commits, dirty tree, PRs, CI — read live from `git` and `gh`. Authorization is GitHub's own `viewerPermission`; **no invented roles.**

THE LOOK-IN

While the agent works, a panel shows the screenshots and the actions it took. **Seeing is the trust model.**

ZERO TELEMETRY • ZERO ADDED NETWORK CALLS

LIVE · THE SHELL

Watch a turn stream

The real app on a canned bridge. Type anything — thinking, tool calls with status, and per-turn token counts stream in.

[LIVE DEMO — SEE SCREEN RECORDING]

L I V E • T H E E N G I N E , N O S H E L L

Same harness, headless

One engine, many doors: terminal, CI, ACP behind another client. The desktop never became a dependency of the thing that does the work.

[L I V E D E M O – S E E S C R E E N R E C O R D I N G]

THE BUILD • REPRODUCIBLE ON PURPOSE

One source of truth, or it does not ship

THE SITE

The published layout is one script. CI calls it; the tests call it. **What deploys and what we verify are the same code path.**

THE BRAND

Logo masters promote to every consumer through one sync script; CI fails on drift. **A stale copy once shipped; now it cannot.**

THE PACKET

The send-one-file demo is generated and checked — **verified to render offline with the network blocked.**

THE HONEST LIMIT

Reproducibility is a discipline, not a property you get for free. Each of these was a real drift that shipped once before the guard existed. **The guard is the product.**



WHAT THIS MEANS FOR THE ROOM

Durable because it is watchable

*A swappable engine under a thin shell. A protocol other clients already speak. A security floor and a test matrix that fail the build on drift. And a local loop — **your machine, your metal, nothing upstream can switch it off.***

THE MOAT IS NOT A FEATURE

It is the habit: every claim in this deck traces to a test, a file, or a live page. **Verify any of it.**

THE ASK NARROWS

A pilot, or a campus door. Either way the substrate is here and inspecting it takes an afternoon.

TOGETHER WE ARE BETTER — ALWAYS